A ChaRNABERT-style model for human DNA

GBST tokenization + bidirectional BERT encoder, on the DNA-xLSTM code base

Report · Omar R.A Mohamed · 17 October 2026

Code: https://github.com/OMAR622-ai/dna-xlstm-charna (branch `charna-gbst`, private)

Summary. The ChaRNABERT architecture — a learnable tokenizer (GBST) in front of a bidirectional BERT encoder — was implemented in the DNA-xLSTM code base, tested (24 automatic tests) and trained on 10.5 billion bases of the human genome (hg38) with the DNA-xLSTM protocol. On unseen test DNA, the final model guesses **56.4%** of hidden bases correctly (random guessing 25%, always the most common base 29.4%), with a loss of **0.975** — in the range of the published 2 M DNA models. A same-size model without GBST (Transformer++) reached 56.3% and 0.977: GBST-BERT was ahead during most of the training but only 0.1 percentage points at the end, too little to call GBST better after one run each. Inside the model, GBST learned to put about 98% of its weight on single bases, so on human DNA it behaves almost like reading one base at a time. No benchmark (downstream task) results yet.

Contents

1 Goal 1

2 How the ChaRNABERT model works 1

3 How we apply ChaRNABERT to DNA 3

4 What was done, step by step 3

5 Results 5

6 Prediction tasks: the Nucleotide Transformer benchmark 8

7 Differences from the papers 10

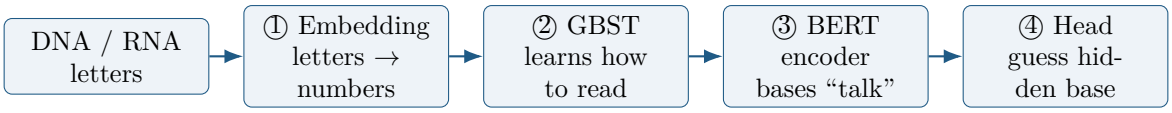
8 Next steps 10

1 Goal

The project asks whether a *learned* way of reading DNA helps a small model. ChaRNABERT (Morales-Pastor et al., 2024) showed this for RNA. We take its architecture, train it on the human genome with the short-context protocol of the DNA-xLSTM paper (Schmidinger et al., 2024), and compare it with models from that paper. The plan has three parts: (1) pretraining on hg38 with masked language modelling; (2) evaluation on held-out DNA (accuracy and loss); (3) later, the 18-task Nucleotide Transformer (NT) benchmark. Parts (1)–(2) are done; part (3) is ready but postponed.

2 How the ChaRNABERT model works

ChaRNABERT is an RNA "foundation model" from Nostrum Biodiscovery (arXiv:2411.11808). Its name combines *Cha*racter (single letters), *RNA* and *BERT*. Its central idea: most models fix in advance how a sequence is cut into "words" (single bases, codons, k-mers); ChaRNABERT lets the model **learn how to cut, separately at every position**.



① Embedding — letters become numbers. Each letter is replaced by a learned list of numbers (256 in our model), like a profile describing that nucleotide.

② GBST (Gradient-Based Subsequence Tokenization) — learning how to read.

1. *Smooth*: each base is mixed a little with its neighbours (a short filter).
2. *List candidates*: at every position, every block of 1, 2, 3 or 4 bases that contains the position. Blocks of the same size start at different places, like reading frames ("offsets"). This gives 10 candidates per position (Table 1).
3. *Score*: a small learned scorer rates each candidate ("how useful is this reading here?"); the scores become weights that add up to 1.
4. *Make the choice consistent*: positions with similar weight patterns pull towards each other, so the "reading style" is consistent along the sequence (ChaRNABERT Eq. 6).
5. *Blend*: each base's output is the weighted mix of its 10 candidates. There is one output per base — ChaRNABERT removes Charformer's shortening step, because many tasks need single-base detail.

Biology analogy: a ribosome reads triplets, a transcription factor a 6–10-base motif, a methylase CpG pairs. GBST can learn to read "like a ribosome" in one place and "like a methylase" in another, without being told which.

Table 1: The 10 GBST candidates for position 3 (the first T) of `ACGTTGCA`.

Block size	Candidate blocks that contain position 3
1	`T`
2	`TT` (positions 3–4), `GT` (2–3)
3	`TTG` (3–5), `GTT` (2–4), `CGT` (1–3)
4	`TTGC` (3–6), `GTTG` (2–5), `CGTT` (1–4), `ACGT` (0–3)

③ BERT encoder — bases talk to each other. A stack of identical blocks (4 in our model). In each block, *attention* lets every base gather information from every other base, in both directions (*bidirectional*); then a small *feed-forward* network processes each position. The paper adds RoPE (encodes distances between bases), QKNorm (keeps training stable), SwiGLU (a better feed-forward type) and FlashAttention 2 (faster attention).

④ Prediction head. For every position, a score for each possible letter.

Learning: the fill-in-the-blank game. In *pretraining*, 15% of the bases are hidden and the model must guess them from the context (*masked language modelling*, MLM). To do this well it must learn motifs, codon patterns, pairing rules — without any labels. The paper also tested *UL2*, which additionally hides longer stretches. ChaRNABERT was trained on 62 million RNA

sequences (31 M non-coding from RNAcentral, 31 M coding from RefSeq), with models of 8 M to 650 M parameters. Afterwards it is *fine-tuned* on specific labelled questions.

What the paper found. GBST gave a lower loss than single-letter reading in most settings (e.g. 0.498 vs. 0.508 for the 5 M model, Table 2); performance saturated around 30–50 M parameters; the 8 M model matched or beat much larger models on several tasks, e.g. RNA–protein binding F1 0.833 vs. 0.831 for the 650 M RiNALMo (Table 5).

3 How we apply ChaRNABERT to DNA

The model stays the same; the data and the setting change.

1. **DNA instead of RNA.** The human genome hg38 (about 3 billion letters), in pieces of 1,024 bases. Alphabet A, C, G, T (N = unknown, ignored).
2. **Letters to numbers.** A = 7, C = 8, G = 9, T = 10 (the repository's tokenizer).
3. **The same game.** 15% of the bases are hidden and must be guessed; with 50% chance the reverse-complement strand is shown instead, so the model learns that both strands are equivalent.

Real:	`A C G T T G C A A G T C`	
Input:	`A C ? T T G C A A ? T C`	
Answer:	`. . G . . . . . . G . .`	(dots: not hidden, not graded)

4. **The same architecture**, GBST + BERT encoder, but small: 2.1 million parameters, the size of the DNA-xLSTM paper's models, so the comparison is fair.
5. **Train, test, compare.** Train on millions of DNA pieces; test on DNA never seen in training (accuracy of the guesses); compare with the same encoder *without* GBST (Transformer++). If ours guesses better, GBST helps on DNA.

Table 2: ChaRNABERT paper vs. this project.

	ChaRNABERT paper	This project
Data	RNA (RNAcentral, RefSeq)	Human DNA (hg38)
Letters	A, C, G, U	A, C, G, T
Model	GBST + BERT	GBST + BERT (same)
Training game	Guess hidden bases (MLM; UL2 tested)	Same (MLM; UL2 available as an option)
Model size	8 M – 650 M parameters	2.1 M (to compare with DNA-xLSTM)
Piece length	up to 8,190 bases	1,024 bases (DNA-xLSTM protocol)
Later tasks	BEACON RNA benchmark	Nucleotide Transformer DNA benchmark

4 What was done, step by step

1. **Read the code base and the four papers** (Bio-xLSTM, ChaRNABERT, Charformer, UL2); every setting was taken from a paper table or section, or marked as our own choice.

2. **Server set-up.** JKU server `student01`: 4× NVIDIA GTX 1080 Ti, shared with another user. Installed a Python environment that works on these older GPUs (PyTorch 2.2 + CUDA 12.1); newer PyTorch versions no longer support them. Copied the genome (3.1 GB) and checked it with an md5 checksum. Set up key-based SSH login.
3. **Implemented the model** (`models/charna/`): GBST (`gbst.py`), the bidirectional encoder and prediction head (`modeling_charna.py`), settings (`configuration_charna.py`), and registered it in the training program. Existing models behave exactly as before.
4. **Implemented UL2 masking** as an option (`src/dataloaders/utils/ul2.py`).
5. **A launcher** (`scripts_charna/train_charna.py`) so the repository's training program runs on GPUs that cannot install two optional packages (mamba_ssm, flash_attn).
6. **Scripts**: pretraining (`run_pretrain_charna.sh`), benchmark fine-tuning, results table, loss plot, accuracy evaluation (`evaluate_mlm.py`), data download with checksum, a small genome sample for GitHub, result collection.
7. **Tests**: 23 automatic tests (Section 5); a check that the model can memorise one batch; a 3-step trial on 4 GPUs; a 48-step smoke run of both models.
8. **Matched the paper exactly** where possible (after review on 4 October): added the "pointwise" layer that ChaRNABERT's parameter table (Table 9) implies; switched the learning rate to ChaRNABERT's 5e-4 (Table 2); restored UL2 as an option.
9. **Bugs found and fixed**: (a) loading pretrained GBST weights into the benchmark code crashed; (b) the benchmark data loader mixed all 18 tasks together (461,850 examples instead of e.g. 5,509) — a bug in the original repository caused by a change on Hugging Face; (c) a results file wrote values under wrong column names; (d) the GitHub genome sample first held only 1/8 of each region; (e) smaller script issues (wrong exit status in a log, a path error).
10. **Small experiment** (4 October): both models trained for 500 steps (≈0.5 billion bases); then accuracy evaluation on 2,048 held-out sequences. The first full run was stopped to get results sooner; the benchmark was postponed.
11. **Learning-rate check** (4–5 October): four short GBST-BERT runs; 2e-3 was best.
12. **Full training** (5–6 October): GBST-BERT with 2e-3, then Transformer++, 10,000 steps each; moved from server student01 to student07 (faster GPUs, same files) at step 3,000.
13. **Data-loading bug** (5 October): found after a crash, measured, fixed with a test; GBST-BERT resumed from step 3,000, Transformer++ restarted.
14. **Final evaluation and analysis** (6 October): accuracy of both final models; what block sizes GBST uses; all models saved on GitHub.
15. **Prediction tasks** (13–17 October): both models fine-tuned on the 18 tasks of the Nucleotide Transformer benchmark, 10 repeats each, as in the Bio-xLSTM paper (Section 6).
16. **Documentation and GitHub**: project README (front page), a beginner's explanation (`docs/EXPLAINED.md`), 50 questions and answers (`docs/QA_50.md`), one page per run (`charna_project/runs/`), everything in one folder (`charna_project/`: code, data that fits in git, models, results), and this report.

Main settings (all for GBST-BERT; source in brackets):

Model	256 numbers per base, 4 encoder blocks, 8 attention heads, 2,107,392 parameters [Bio-xLSTM Table A3]
GBST	blocks of 1–4 bases with offsets, sum pooling, calibration, no shortening [ChaRNABERT §3.2]
Masking	15% of bases; of these 80% `[MASK]`, 10% random, 10% unchanged [Bio-xLSTM App. C.5]
Batch	1,024 sequences × 1,024 bases = 1,048,576 bases per step [Bio-xLSTM App. C.5]
Learning rate	2e-3, best of 5e-4 [ChaRNABERT Table 2], 1e-3, 2e-3, 5e-3 in a short check; warmup 10% of steps, cosine decay [Bio-xLSTM Table A4]
Optimizer	AdamW, $\beta = (0.9, 0.95)$, weight decay 0.1 [Bio-xLSTM Table A4]
Length of training	small experiment 500 steps (0.5 B bases); full run 10,000 steps (10.5 B)
Comparison model	Transformer++-2M, learning rate 1e-2 [Bio-xLSTM Table A3]

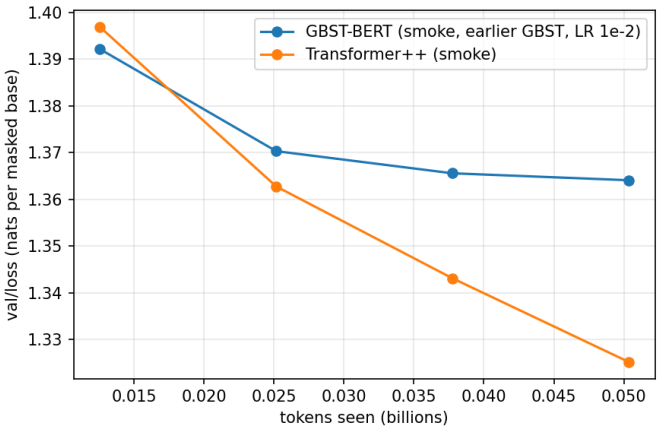
5 Results

Automatic tests: 23 of 23 pass. They check, among others, that GBST matches a slow reference implementation of the paper's equations, that it has the weight count of the paper's Table 9, that the model looks in both directions, that the loss counts only hidden bases, that about 15% of bases are hidden, that UL2 never leaks the answer, and that pretrained weights load correctly into the benchmark code.

Memorising one batch: passed. The loss fell below 0.05 after 164 steps (MLM) and 230 steps (UL2), so the model is able to learn.

Smoke run (48 steps, 50 million bases, an earlier GBST version with learning rate 1e-2). Loss is in nats per hidden base; 1.386 = random guessing among 4 bases.

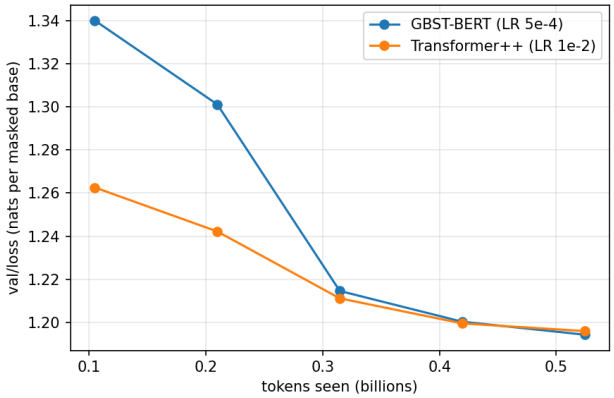
Bases	GBST-BERT	Transformer++
13 M	1.392	1.397
25 M	1.370	1.363
38 M	1.366	1.343
50 M	1.364	1.325



Both models learn, but this run only checked that everything works: it was very short and used the older GBST and learning rate. A test of the accuracy script on these smoke models showed why accuracy needs reference values: the smoke GBST model reached 29.1% only by always answering "A" (the most common hidden base), i.e. no better than no model; Transformer++ reached 32.8%.

Small experiment (finished 4 October, 22:05). Both models trained for 500 steps (≈0.52 billion bases) with the settings of Section 4. Validation loss during training (2% of the validation set):

Step	Bases	GBST-BERT	Transformer++
100	0.10 B	1.3399	1.2626
200	0.21 B	1.3011	1.2422
300	0.31 B	1.2146	1.2112
400	0.42 B	1.2004	1.1996
500	0.52 B	**1.1944**	1.1960



Accuracy test. The final models predicted the hidden bases of 2,048 test sequences they had never seen (311,767 hidden bases; both models got exactly the same hidden positions). Validation-split values in brackets.

Model	Loss	Accuracy	A	C	G	T
GBST-BERT	1.1944 (1.1945)	**43.94%** (44.12%)	53.9%	32.0%	36.1%	47.9%
Transformer++	1.1958 (1.1955)	43.77% (43.95%)	51.0%	33.0%	31.5%	52.8%
Random guess	1.3863	25.00%	25%	25%	25%	25%
Always “A” (most common)	–	29.42% (29.97%)				

What this means.

- *Both models learned real DNA patterns*: 44% accuracy, well above random guessing (25%) and above always guessing the most common base (29–30%); loss 1.19 versus 1.37 for a model that knows only the base composition.
- *GBST-BERT starts slower, then catches up*: it first has to learn *how* to read before it can use it. At the end it is very slightly ahead (+0.17 percentage points accuracy on both splits). With a single training run per model, this is too small to call a winner.
- *Different strengths per base*: GBST-BERT predicts G better (36% vs. 31.5%) and T worse (48% vs. 53%).
- *Room to improve*: the published 2 M models reach about 0.96–1.03 test loss after 2–10 billion bases (Bio-xLSTM Fig. 2). Both curves here are still falling at 0.5 billion bases.

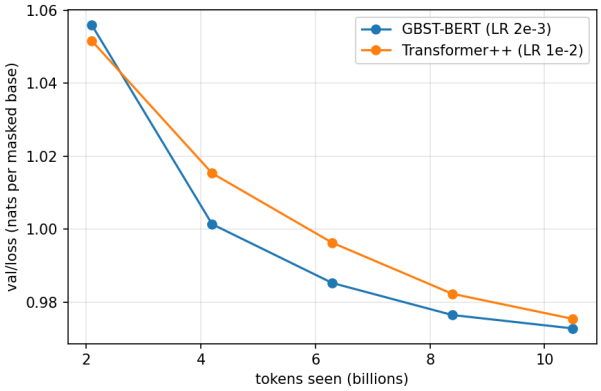
Learning-rate check (5 October)

ChaRNABERT’s 5e-4 comes from an RNA model with another batch size, so four GBST-BERT runs with the small-experiment setup (500 steps) were compared by *validation* loss (the test split was not used for the choice): 5e-4: 1.194, 1e-3: 1.187, **2e-3: 1.154**, 5e-3: 1.160. The full run used 2e-3. Transformer++ kept its paper value (1e-2, Bio-xLSTM Table A3) and was not re-tuned.

Full training (10.5 billion bases)

Both models trained for 10,000 steps (10.5 billion bases, about 4.5 passes over the 2.33 billion unique training bases). Validation loss during training (12.5% of the validation set):

Step	Bases	GBST-BERT	Transformer++
2,000	2.1 B	1.0559	1.0516
4,000	4.2 B	**1.0014**	1.0153
6,000	6.3 B	**0.9853**	0.9963
8,000	8.4 B	**0.9765**	0.9823
10,000	10.5 B	**0.9728***	0.9755



* After resuming, the training program counted one step ahead, so GBST-BERT's last scheduled check fell on step 10,001 and was skipped. It was measured afterwards on the final model with exactly the same settings (validation 0.9728, test 0.9670).

Final accuracy test. Both final models predicted the hidden bases of the same 2,048 unseen test sequences (311,767 hidden bases, identical masks). Validation split in brackets.

Model	Loss	Accuracy	A	C	G	T
GBST-BERT	**0.9753** (0.9689)	**56.44%** (56.80%)	61.8%	48.5%	48.8%	62.1%
Transformer++	0.9774 (0.9728)	56.33% (56.65%)	61.3%	48.9%	48.9%	61.9%
Random guess	1.3863	25.00%	25%	25%	25%	25%
Always "A" (most common)	–	29.42% (29.97%)				

What GBST learned

Measured on 8 sequences from different chromosomes, the share of GBST's weight on blocks of 1 / 2 / 3 / 4 bases was 72 / 22 / 5 / 1% after 500 steps at learning rate 5e-4, but 98 / 2 / 0 / 0% at 2e-3 and **98 / 2 / 0 / 0% in the final model**. On human DNA, GBST thus learned to read almost only single bases (each still mixed with its neighbours by the convolution before the blocks).

A data-loading bug, found and fixed

On 5 October the full GBST-BERT run crashed when the data loader returned an empty sequence. Cause: in the repository's genome reader, the parallel reader processes shared one open file and its read position, so they sometimes read the wrong bases. Measured: 0.35% of sequences with 2 readers per GPU (all runs on the first server), 13–33% with 12. Wrong reads stayed within their data split (no test DNA in training), and the accuracy tests read without parallel readers. The file is now opened separately in each reader (with a test: 841 of 1,024 sequences wrong without the fix, 0 with it); GBST-BERT was resumed from its last checkpoint before the problem (step 3,000) and Transformer++ was trained from scratch with the fix.

Conclusions

- Both 2 M models learned human DNA well (56% accuracy, loss ≈0.975), in line with the published models of the same size.
- GBST-BERT was ahead for most of the training, but at the end the difference is 0.1 percentage points — too small to say that GBST helps, with one run per model.

- GBST learned to use mostly single bases, which explains why the two models end so close.
- Not fully controlled: only GBST-BERT's learning rate was tuned, and the two models also differ in normalization. The same model with GBST replaced by a plain lookup table would isolate the effect of GBST.

6 Prediction tasks: the Nucleotide Transformer benchmark

A pretrained model is only useful if it helps with real biological questions. As in the Bio-xLSTM paper, we fine-tuned both final models on the 18 tasks of the Nucleotide Transformer benchmark: 10 tasks ask whether a DNA piece carries a histone mark, 2 ask about enhancers, 3 about promoters and 3 about splice sites. For each task the model reads a DNA piece, averages its output over all positions (and over both DNA strands) and a small added layer predicts the answer; all weights are trained for 20 epochs, and the epoch with the best validation score is tested. Every task was repeated with **10 seeds** (each with a new training/validation split), and the table reports the mean and the range (best minus worst), exactly like Bio-xLSTM Table 1. Learning rate and batch size per task are the ones Bio-xLSTM chose for its DNA-xLSTM-PH model (Table A5). Exception: on splice acceptors and donors these high learning rates (2e-3, 3e-3) made GBST-BERT collapse to always answering "yes" (F1 0.663 in all 3 first donor runs), so 1e-3 and 5e-4 were also tried (3 seeds) and the best by *validation* score was used.

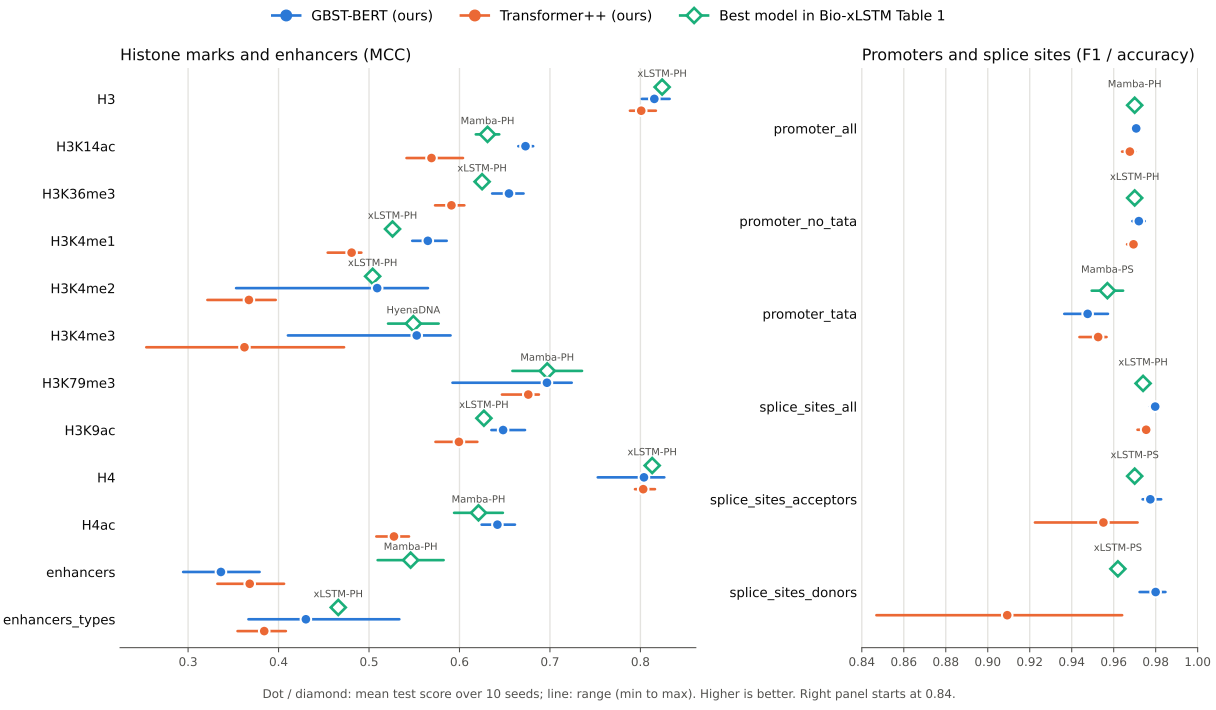


Figure 1: Test scores on the 18 tasks (higher is better). Dot/diamond: mean over 10 seeds; line: range. Green: the best of the five models in Bio-xLSTM Table 1 for that task.

Task	Metric	Best paper model	its score	GBST-BERT	Transformer++
H3	MCC	xLSTM-PH	**0.824 ± 0.010**	0.815 ± 0.031	0.801 ± 0.029
H3K14ac	MCC	Mamba-PH	0.631 ± 0.026	**0.673 ± 0.017**	0.569 ± 0.062
H3K36me3	MCC	xLSTM-PH	0.625 ± 0.010	**0.655 ± 0.035**	0.591 ± 0.032
H3K4me1	MCC	xLSTM-PH	0.526 ± 0.001	**0.565 ± 0.038**	0.481 ± 0.037
H3K4me2	MCC	xLSTM-PH	0.504 ± 0.012	**0.509 ± 0.212**	0.367 ± 0.076
H3K4me3	MCC	HyenaDNA	0.549 ± 0.056	**0.553 ± 0.180**	0.362 ± 0.219
H3K79me3	MCC	Mamba-PH	**0.697 ± 0.077**	**0.697 ± 0.132**	0.676 ± 0.041
H3K9ac	MCC	xLSTM-PH	0.627 ± 0.008	**0.648 ± 0.037**	0.599 ± 0.046
H4	MCC	xLSTM-PH	**0.813 ± 0.008**	0.804 ± 0.074	0.803 ± 0.022
H4ac	MCC	Mamba-PH	0.621 ± 0.054	**0.642 ± 0.037**	0.528 ± 0.036
enhancers	MCC	Mamba-PH	**0.546 ± 0.073**	0.336 ± 0.085	0.368 ± 0.074
enhancers_types	MCC	xLSTM-PH	**0.466 ± 0.011**	0.430 ± 0.167	0.384 ± 0.053
promoter_all	F1	Mamba-PH	0.970 ± 0.004	**0.971 ± 0.003**	0.968 ± 0.006
promoter_no_tata	F1	xLSTM-PH	0.970 ± 0.001	**0.972 ± 0.006**	0.969 ± 0.004
promoter_tata	F1	Mamba-PS	**0.957 ± 0.015**	0.948 ± 0.021	0.953 ± 0.013
splice_sites_all	Accuracy	xLSTM-PH	0.974 ± 0.004	**0.980 ± 0.004**	0.976 ± 0.006
splice_sites_acceptors	F1	xLSTM-PS	0.970 ± 0.005	**0.978 ± 0.009**	0.955 ± 0.049
splice_sites_donors	F1	xLSTM-PS	0.962 ± 0.004	**0.980 ± 0.012**	0.909 ± 0.117

Mean ± (max − min) over 10 seeds; bold = best of all seven models (the five paper models and ours).

What the prediction results show

- GBST-BERT has the best mean on **13 of 18 tasks** (Bio-xLSTM's best model, xLSTM-PH, on 4): 8 of 10 histone tasks (one a tie), 2 of 3 promoter tasks and all 3 splice tasks.
- GBST-BERT beats Transformer++ — the same size, same pretraining, but no GBST — on **16 of 18 tasks**. Pretraining loss could not separate the two models; these prediction tasks clearly can, and they favour GBST.
- Enhancers are the weak spot of both our models (MCC 0.34 / 0.43 vs. 0.55 / 0.47 for the best paper models).
- Some GBST-BERT runs trained poorly (wide ranges on H3K4me2, H3K4me3, H3K79me3), which is why 10 repeats matter.
- Not fully controlled: the paper tuned learning rate and batch size for each of its models; we reused its DNA-xLSTM-PH values (only the two splice tasks were re-checked).

7 Differences from the papers

Paper	This project	Reason
FlashAttention 2, bf16 numbers	standard attention, fp32	Our GPUs (GTX 1080 Ti, RTX 2080 Ti, Quadro GV100) support neither; the result is the same, only slower
Eq. 3: offsets written as $X_{i+o:i+o+b}$	blocks that contain position i	Read literally, a position would get blocks that do not include it; the text (reading frames) describes our version
Eq. 9: SwiGLU written as $\sigma(xW_1) \odot \text{swish}(xW_2)$	standard SwiGLU	The text cites the standard SwiGLU; the printed formula is a different function
Eq. 1: depthwise convolution	+ a 256×256 pointwise layer	The paper's Table 9 counts $d \cdot M + d^2$ weights, which implies it
Maximum block size not stated	4	As in Charformer's figure (our choice)
RNA, 2–8 k bases, 8–650 M parameters	DNA, 1,024 bases, 2 M	Comparability with DNA-xLSTM (project plan)
Repository Transformer++ script: LR 8e-3, hidden size 1,024	LR 1e-2, hidden size 320	We follow the Bio-xLSTM paper's Table A3

Uncertain points: the block size (4); our reading of Eq. 3 and Eq. 9; the inferred pointwise layer; whether learning rate 5e-4 (best for ChaRNABERT's 5 M RNA model) is also good for a 2 M DNA model with much larger batches (our check found 2e-3 better); each model was trained once, so run-to-run variation is not measured.

8 Next steps

1. A clean test of GBST: the same model with GBST replaced by a plain lookup table (`TOKENIZER=embedding`), same learning rate and settings — about 14 hours.
2. Optional: MLM vs. UL2, with/without strand flipping, repeated runs with other seeds.

References

- Morales-Pastor et al. (2024). Character-level Tokenizations as Powerful Inductive Biases for RNA Foundational Models (ChaRNABERT). arXiv:2411.11808.
- Schmidinger et al. (2024). Bio-xLSTM: Generative modeling, representation and in-context learning of biological and chemical sequences. arXiv:2411.04165.
- Tay et al. (2022). Charformer: Fast Character Transformers via Gradient-based Subword Tokenization. ICLR 2022, arXiv:2106.12672.
- Tay et al. (2023). UL2: Unifying Language Learning Paradigms. ICLR 2023, arXiv:2205.05131.
- Devlin et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL 2019.